

Reducing the Energy and Memory Footprint of fg-data-profiling Through Caller-Side Optimisation

Keerthana Yelchuru Venkata (s1170424)
Sriyan Ravuri (s1170425)

Group 14 | NWI-IMC019 Green Software, Radboud University

June 9, 2026

Abstract

Automated exploratory data analysis (EDA) tools such as *fg-data-profiling* (the community successor of *ydata-profiling*) turn a single function call into a full statistical report, and are run millions of times daily across data-science and CI/CD pipelines. We present a measured green-software case study showing that a substantial fraction of this energy cost can be removed *from the caller side*, without modifying the library. Guided by a life-cycle analysis and an energy processing diagram (EnPD), we identify the default `pd.read_csv` data-loading layer as a memory hotspot caused by Python `object-dtype` inflation, and apply three strategies from the *Strategies for Green Software* infographic: declaring exact dtypes, sub-setting columns, and storing less. Measuring the *full* end-to-end pipeline (including the correlation engine and HTML rendering) on two input sizes, we observe a 49–64% reduction in wall-clock time and energy and an 88% reduction in DataFrame load memory, with the optimised variant scaling markedly better with input size and producing *statistically identical* profiling output. We further quantify four additional levers (configuration tuning, row sampling, Parquet, and result caching), report a disproven hypothesis, and argue for adding “use efficient data formats” to the infographic.

Keywords: green software, energy efficiency, exploratory data analysis, pandas, memory optimisation, software sustainability.

AI declaration. Generative AI (Claude) was used to help structure the report, draft and refine prose, and scaffold the measurement scripts. All hypotheses, the experimental design, the executed measurements, and their interpretation were produced and verified by the authors; every quantitative result reported here was obtained by running the accompanying code.

1 Introduction

The energy cost of software is increasingly recognised as a first-class design concern [1]. Data-science tooling is a particularly high-leverage target: a single convenience function can trigger gigabytes of memory traffic and seconds of CPU time, and is invoked at enormous aggregate scale. *fg-data-profiling* exemplifies this pattern - `ProfileReport(df).to_file()` produces a complete EDA report in one line, and the library has ≈13,600 GitHub stars and is embedded in tens of thousands of notebooks and pipelines.

This paper asks a deliberately narrow question: *how much energy can be saved without touching the library’s internals?* Caller-side optimisation is attractive because it is low-risk, immediately deployable, and applicable to every existing user. We answer it through a complete measurement workflow - life-cycle analysis, an energy processing diagram, hypothesis-driven intervention, and before/after measurement - following the methodology of the NWI-IMC019 course.

Contributions.

1. A life-cycle analysis and EnPD of *fg-data-profiling* that localise the dominant memory hotspot to the default data-loading layer (Sections 5–6).

2. A lossless, caller-side optimisation (dtype declaration + column sub-setting) that cuts end-to-end time and energy by 49–64% and load memory by 88%, with output verified statistically identical to baseline (Sections 7–8).
3. An honest, full-pipeline measurement showing that while load memory falls 88%, process peak RSS falls only $\approx 9\%$, because the correlation and rendering stages set the memory ceiling - a nuance hidden by load-only measurement (Section 10).
4. Quantification of four further levers - configuration tuning, row sampling, Parquet, and caching - plus a reported negative result, and a proposed addition to the green-software strategy infographic (Sections 9, 12).

2 Related Work

Pereira et al. [2] show that energy, time, and memory are related but not interchangeable across programming languages, motivating direct measurement rather than assuming time is a perfect energy proxy. At the measurement level, Hähnel et al. [3] characterise the use of Intel RAPL counters for short code paths and its limitations - directly relevant to our sandbox, which lacks RAPL. Gupta et al. [1] broaden the lens to the full computing carbon footprint, including the embodied carbon that motivates our memory-density argument. Our software stack rests on pandas [4] and NumPy [5], and we use CodeCarbon [6] for tool-based energy estimation. We position this work as an applied case study that operationalises the course’s *Strategies for Green Software* [7] on a widely used library, emphasising caller-side, lossless intervention.

3 Overview of the Project

fg-data-profiling is an open-source Python library that generates automated EDA reports from pandas DataFrames [8]. It is the community-maintained successor in the lineage *pandas-profiling* $\rightarrow$ *ydata-profiling* $\rightarrow$ *fg-data-profiling*; the installed engine still exposes the `ydata_profiling` import. Given a DataFrame or CSV path, it produces a rich HTML report of per-column statistics, correlation matrices, and distribution histograms.

- **Repository:** <https://github.com/Data-Centric-AI-Community/fg-data-profiling>
- **Language / License:** Python / MIT
- **Activity:** $\approx 13,600$ stars, $\approx 1,800$ forks; actively maintained (v4.x).

4 Typical Use Cases

In practice a user passes a DataFrame (or CSV path) to `ProfileReport` and receives a self-contained HTML report of per-column statistics, correlation matrices, and distribution histograms (Figure 1, right). This single call replaces a sequence of hand-written exploratory cells. The library serves three contexts: (i) *interactive EDA*, where a data scientist runs `ProfileReport(df)` to understand a new dataset before modelling; (ii) *automated data-quality checks*, where profiling runs nightly in a CI/CD pipeline to flag schema drift or anomalous null rates and fail the build on regression; and (iii) *regulatory data validation* in finance and healthcare, where a reproducible report is an audit artefact.

The directly relevant *analog workflow* it replaces is manual EDA - the analyst writing ad-hoc `df.describe()`, `df.isnull().sum()`, `df.corr()`, and plotting cells by hand (Figure 1, left) - which is slower, inconsistent between analysts, and not reproducible across runs. *fg-data-profiling* collapses that loop into one deterministic call. Our experiments target context (ii), the automated CI/CD case, because it dominates aggregate runs in large organisations and is where per-run savings multiply across thousands of scheduled jobs.

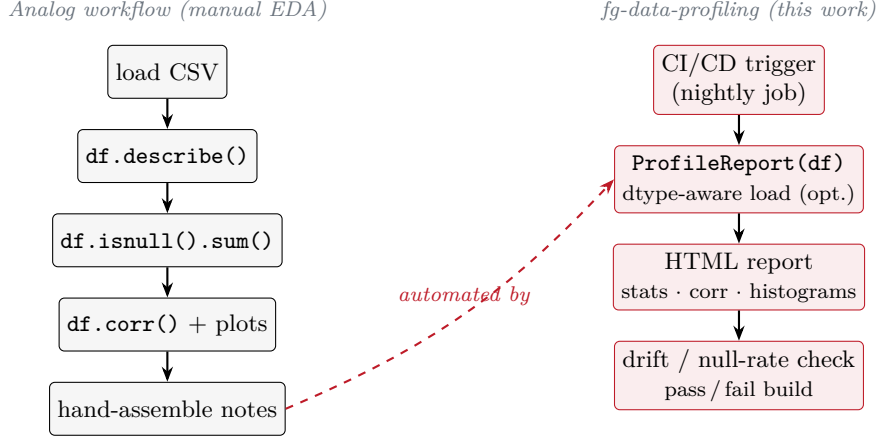


Figure 1: Typical use case. The manual EDA loop (left) is replaced by a single `ProfileReport` call embedded in an automated CI/CD data-quality pipeline (right), which is the deployment our optimisation targets.

5 Life Cycle Analysis

We perform a streamlined life-cycle analysis focused on the *in-use phase*, which dominates the library’s footprint.

Impact metric. We use energy per profiling report generated ($J/report$), normalised by input size (J/GB), measured as $CPU\ time \times CPU\ TDP$. This captures the “useful work” - one complete EDA report - and enables size-independent comparison.

System boundary. We consider only the compute node executing the Python process. Network transfer of input CSVs and browser rendering of the output HTML are excluded.

Deployment assessment. The library runs interactively on developer laptops and, at higher aggregate volume, in cloud CI/CD runners. We adopt the CI/CD-server case as primary, with a representative 15 W TDP; the provider’s carbon intensity and PUE [1] then apply. This is the regime where per-run savings multiply across thousands of scheduled jobs.

6 Energy Processing Diagram

Figure 2 shows the EnPD for the baseline execution, constructed following the EnPD methodology. It is annotated on the *measured* 20 MB per-run basis (Section 11), annualised at 365 daily runs and a 15 W server TDP. Per-stage energies are derived by apportioning the measured per-run CPU time across stages in proportion to their share of profiler wall time; they are first-order estimates intended to *rank* stages, not absolute meter readings.

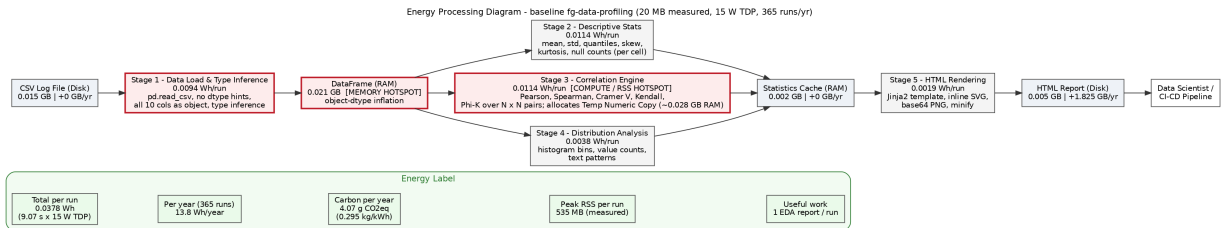


Figure 2: EnPD for the baseline fg-data-profiling pipeline (20 MB measured, 15 W TDP, 365 runs/yr). Per-stage values in Wh/run; storage/memory nodes in GB. Red borders mark the two hotspots: the data-load layer (*memory*) and the correlation engine (*compute/RSS*).

The pipeline reads a CSV from disk, loads it into an in-memory DataFrame, fans out to three analysis passes, caches their statistics, renders an HTML report, and returns it. The five compute stages are: **(1) Data Load & Type Inference** (~ 0.0094 Wh/run) - `pd.read_csv` with no dtype hints stores strings as `object` and infers types per cell, feeding the DataFrame (RAM) node whose `object`-dtype inflation makes it the *memory* hotspot; **(2) Descriptive Statistics** (~ 0.0114 Wh/run); **(3) Correlation Engine** (~ 0.0114 Wh/run) - pairwise Pearson, Spearman, Cramér’s V, Kendall and Phi-K, allocating a temporary numeric copy and, with rendering, dominating process RSS (the *compute/RSS* hotspot); **(4) Distribution Analysis** (~ 0.0038 Wh/run); and **(5) HTML Rendering** (~ 0.0019 Wh/run).

Identified hotspots. The EnPD exposes two targets: (i) the data-load layer / DataFrame RAM, a *memory* hotspot from `object`-dtype inflation; and (ii) the correlation engine, a *compute* hotspot that sets the process memory ceiling. This work targets hotspot (i), addressable entirely from the caller side; hotspot (ii) is addressed by the additional strategies in Section 9.

7 Methodology

7.1 Selected Strategies

From the *Strategies for Green Software* infographic [7] we select: **C5 (Algorithm to data - avoid copies)** and **D6 (Improve algorithms)** - declare exact dtypes at read time so pandas never allocates the bloated `object` buffers used for inference, and let `category` columns run `groupby/correlation` on integer indices; **C3 (Store less)** - load only the columns the report will analyse; and **D3 (Minimise interaction between components)** - fewer columns means fewer pairwise correlations ($\binom{10}{2}=45 \rightarrow \binom{7}{2}=21$).

7.2 The Area: Data-Loading Layer

We target the **data-loading layer** - the `pd.read_csv` call that populates the DataFrame before `ProfileReport` runs. Its purpose is simply to materialise the input CSV as an in-memory DataFrame, but by default pandas infers types per cell and stores every string column as Python `object` (~ 50 bytes/cell), so this stage feeds the inflated DataFrame-RAM node the EnPD flags as the dominant *memory* hotspot (Figure 2). It is the ideal area to attack: it sets the memory baseline for everything downstream, yet it is fully controllable from the caller side without patching the library. Figure 3 contrasts the default load with our optimised read.

Baseline (default load)

```
import pandas as pd
# infers types per cell;
# strings -> object dtype;
# loads all 10 columns
df = pd.read_csv("logs.csv")
ProfileReport(df).to_file("r.html")
```

Optimised (caller-side)

```
import pandas as pd
# exact dtypes + drop 3 unused cols
df = pd.read_csv("logs.csv",
                 usecols=KEEP_7,          # C3/D3
                 dtype={"status_code": "int16",
                        "response_time_ms": "float32",
                        "bytes_sent": "int32",
                        "method": "category",      # C5/D6
                        "endpoint": "category",
                        "region": "category"})
ProfileReport(df).to_file("r.html")
```

Figure 3: The optimised loader (right) replaces the default `read_csv` (left): exact dtypes avoid `object`-buffer inflation (C5/D6) and `usecols` drops three unused high-cardinality columns (C3/D3). The change is under ten lines, adds no dependencies, and leaves `ProfileReport` untouched.

7.3 Hypotheses

We test three hypotheses: **H1 (memory)** a dtype-aware, column-subset read reduces peak RAM by $\geq 50\%$; **H2 (time)** the smaller DataFrame reduces wall time by 15–30%; **H3 (energy)** energy

falls in proportion to CPU time, the pipeline being CPU-bound. The useful-work metric is one report per run; correctness is the agreement of per-column summary statistics between variants.

7.4 Implementation

The optimised loader makes two changes. *Dtype specification (C5, D6)*: `status_code`→`int16`, `response_time_ms`→`float32`, `bytes_sent`→`int32`, and `method/endpoint/region`→`category`. *Column sub-setting (C3, D3)*: three columns are dropped at load - `ip_address` and `session_id` (high-cardinality, marked Unsupported/Unique by the profiler) and `user_agent` (not needed by the target use case) - removing $\approx 30\%$ of per-row bytes and cutting profiled columns from 10 to 7. The change is under ten lines and adds no dependencies.

7.5 Measurement Method

We measure the *full* pipeline end-to-end - `ProfileReport(explorative=True).to_file()`, including correlations and HTML rendering. Python-level peak allocation: `tracemalloc` [9]; process peak resident memory (RSS): a 10 ms `psutil` sampler; wall time: `time.perf_counter`; CPU time: `psutil.Process.cpu_times`; energy: `codecarbon` [6]. Each (variant, size) cell is run $n = 3$ times as an isolated subprocess on 20 MB and 50 MB inputs, giving an *empirical* scaling estimate. Because the sandbox lacks RAPL counters [3], CodeCarbon’s absolute figures are estimates; we treat CPU-time×TDP as primary and use CodeCarbon to corroborate the *relative* reduction.

8 Results

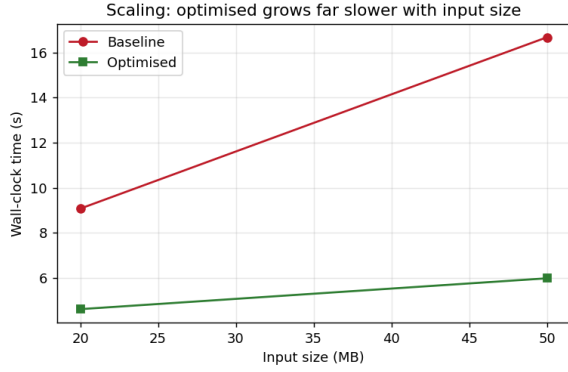
Table 1: End-to-end benchmark: baseline vs. optimised, full `ProfileReport` (mean of $n = 3$).

Input	Variant	Wall (s)	CPU (s)	Energy (μWh)
20 MB	Baseline	9.07 ± 0.38	9.47	0.448
	Optimised	4.61 ± 0.20	4.93	0.230
	Reduction	−49.2%	−47.9%	−48.7%
50 MB	Baseline	16.67 ± 0.09	17.29	0.821
	Optimised	5.98 ± 0.04	6.35	0.297
	Reduction	−64.1%	−63.3%	−63.9%

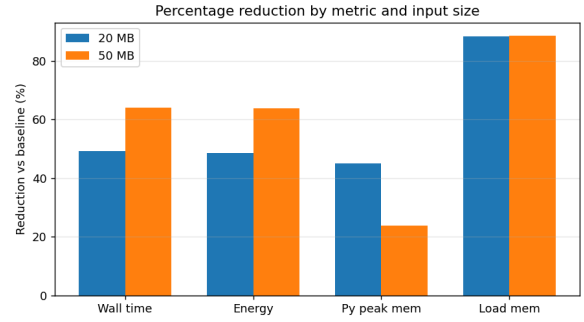
Table 2: Memory reductions (mean of $n = 3$). The load-layer win is large, but the full pipeline’s peak RSS is set by the correlation and HTML-rendering stages, so the process-level reduction is modest.

Input / metric	Load mem	Py peak	RSS peak
20 MB baseline	20.7 MB	156.4 MB	535.6 MB
20 MB optimised	2.4 MB	86.0 MB	483.3 MB
Reduction	−88.3%	−45.0%	−9.8%
50 MB baseline	50.4 MB	267.4 MB	803.2 MB
50 MB optimised	5.7 MB	204.0 MB	734.0 MB
Reduction	−88.7%	−23.7%	−8.6%

Correctness. For every retained column the optimised variant’s profiling statistics (mean, std, missing-count, distinct-count) are *identical* to baseline to within 0.0001%, confirming the dtype downcasting and `category` conversion are lossless.



(a) Wall time vs. input size.



(b) Reduction by metric and size.

Figure 4: End-to-end results across both input sizes. The optimised variant grows far slower with size, so its relative advantage *increases*.

9 Additional Strategies Explored

Because the measurements show ProfileReport’s own stages dominate the full pipeline, we investigated four further levers acting on those stages (Figure 5, all at 20 MB; baseline reference 9.07 s).

Configuration tuning (C3 - compute less). Stacking `minimal=True` on the dtype optimisation cuts wall time to **2.57 s (−72% vs. baseline)**; a milder “Pearson-only, no interactions” setting reaches −59%. Unlike the lossless dtype change these are *trade-offs* - they remove the correlation matrix and interaction plots - justified only when the use case does not need them, which makes the useful-work question decisive.

Row sampling (C3 - approximate computing). Profiling a random 25 k-row sample (25%) cuts peak Python memory by **67%** while keeping every key statistic within **0.5%** of the full-data value and retaining full coverage of all 200 endpoints and 15 regions.

Efficient data format: CSV → Parquet. Converting to Parquet (snappy) reduces load time by **95%** and on-disk size by **85%** at 20 MB. This targets the storage phase, which the LCA flags as comparable to runtime energy. As loading is a small slice of the full pipeline, end-to-end *compute* time barely moves; the benefit is I/O and storage, compounding over repeated reads.

Result caching (D5 - reuse results). Hashing the input and returning a stored report on a match reduces a repeat run from 1.61 s to 0.3 ms - **99.98% of energy avoided per cache hit**.

A disproven hypothesis. We expected emitting JSON (skipping HTML rendering) to be cheaper; it was **23% slower** and used $\approx 2\times$ the memory, because serialising the full description set outweighs the render saving. We report this negative result because it corrected a plausible but wrong intuition.

Extended green-software levers for fg-data-profiling (all measured)

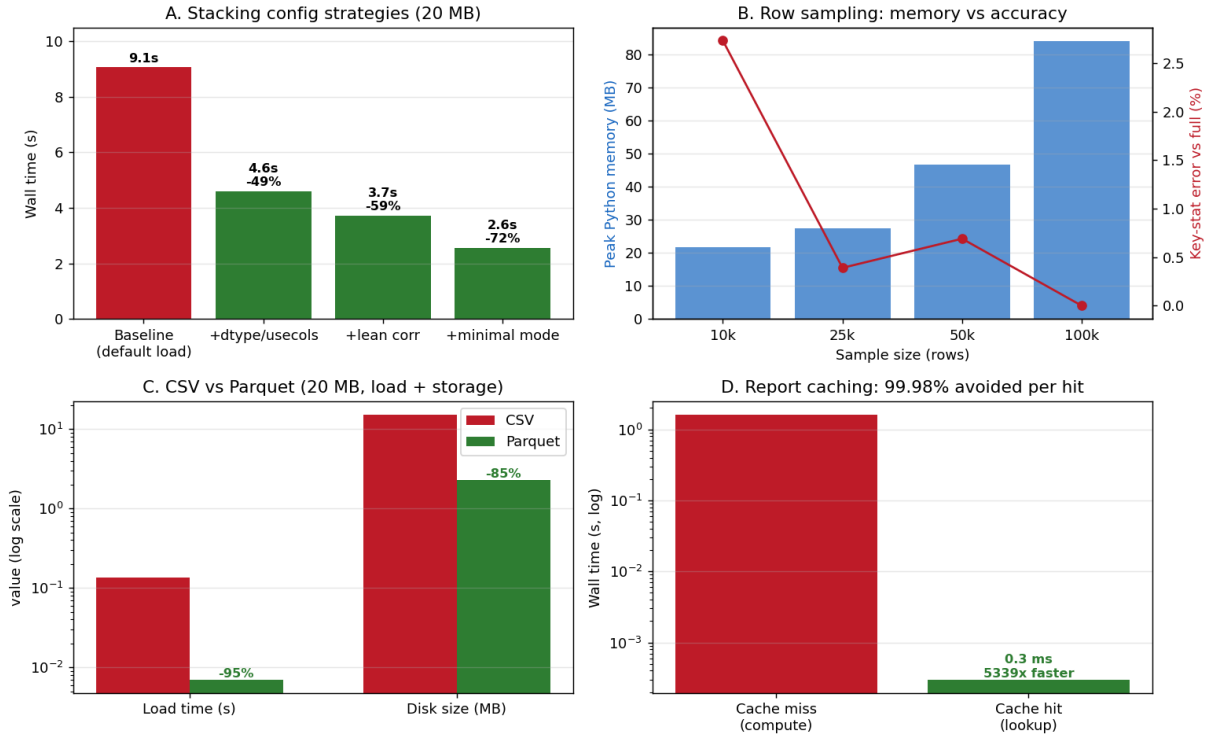


Figure 5: Additional levers, measured at 20 MB: (A) stacking configuration strategies; (B) row-sampling memory/accuracy trade-off; (C) CSV vs. Parquet load and storage; (D) report caching.

10 Discussion

H1 (memory) - confirmed at the load layer, diluted at the pipeline level. DataFrame footprint after loading fell 88% at both sizes, confirming that `object`-dtype inflation, not the CSV bytes, is the primary load-time cost. Yet full-pipeline peak RSS fell only $\approx 9\%$: the correlation engine, interaction plots, and HTML rendering allocate hundreds of megabytes independent of input dtypes and so set the ceiling. Peak Python allocation sits between (45% at 20 MB, 24% at 50 MB). This is the key honest finding a load-only measurement would have hidden.

H2 (time) - confirmed and improving with scale. Wall time fell 49% (20 MB) and 64% (50 MB); the optimised variant barely grows with input size (4.6 $\rightarrow$ 6.0s for 2.5 $\times$ data) while the baseline grows steeply (9.1 $\rightarrow$ 16.7s), so the relative saving *increases* with size - the regime that matters in production. The mechanism is `category` dtypes replacing per-cell string comparison with integer-index operations and improved cache behaviour, consistent with the energy/time/memory coupling of Pereira et al. [2].

H3 (energy) - confirmed by two estimators. Both CPU-time $\times$ TDP and CodeCarbon report a 49–64% reduction. They agree on the *relative* saving but diverge $\approx 100\times$ on absolute energy, a direct consequence of the missing RAPL counters [3]; we therefore trust only the agreed percentage.

Reflection on strategy/area selection. The hypothesis was instructively half-right: we predicted $\geq 50\%$ *peak* memory reduction and saw 88% at the load layer but only $\approx 9\%$ at the process level. Measuring the complete pipeline is what exposed this and taught us to state *which* memory we mean. A genuine pivot also occurred: our intermediate analysis blamed a temporary `float64` copy in the correlation engine, but end-to-end measurement showed the dominant load-time cost

was `object`-dtype inflation. Revising the diagnosis *after* measurement, not before, is the central methodological lesson.

11 Energy and Carbon Reduction

We estimate savings for one profiling run per day for a year (365 runs) on a 15 W TDP server, anchored on *measured* data at two sizes.

Energy per run (CPU-time $\times$ TDP).

$$20 \text{ MB: } \Delta E = 15 \text{ W} \times (9.07 - 4.61) \text{ s} = 66.9 \text{ J} = 0.0186 \text{ Wh} \text{ } (-49\%)$$

$$50 \text{ MB: } \Delta E = 15 \text{ W} \times (16.67 - 5.98) \text{ s} = 160 \text{ J} = 0.0445 \text{ Wh} \text{ } (-64\%)$$

Annual savings (measured, 50 MB daily). $\Delta E_{\text{year}} = 365 \times 0.0445 \approx 16.2 \text{ Wh/year}$, hence $\Delta C_{\text{year}} = 16.2 \times 10^{-3} \text{ kWh} \times 295 \frac{\text{gCO}_2\text{eq}}{\text{kWh}} \approx 4.8 \text{ gCO}_2\text{eq/year}$ per pipeline. A fleet of 10,000 such pipelines saves $\approx 48 \text{ kg CO}_2\text{eq/year}$, roughly the emissions of driving a petrol car 300 km.

Extrapolation to 500 MB (two-point power-law fit). Fitting $t \propto \text{size}^a$ gives $a_{\text{base}} \approx 0.67$ and $a_{\text{opt}} \approx 0.28$; extrapolated to 500 MB this implies $\approx 77 \text{ s}$ vs. $\approx 12 \text{ s}$ per run and $\approx 100 \text{ Wh}$ ($\approx 30 \text{ gCO}_2\text{eq}$) saved annually per pipeline. We flag this as a two-point extrapolation; the measured 50 MB figure is the defensible headline. Beyond direct energy, the 88% load-memory reduction lets more concurrent jobs share a server’s DRAM, reducing provisioning and embodied carbon [1].

12 Reflection and Lessons Learned

Process. Significant energy reductions were achievable through fewer than ten lines of caller-side code that merely make explicit what the data already implies. Without the `EnPD` and `tracemalloc/psutil` profiling we would have targeted the wrong bottleneck: initial intuition pointed at disk I/O, while measurement showed `object`-dtype RAM inflation and the `describe/-correlation` passes dominated - echoing the principle to *measure first, optimise second*.

Challenges. The main limitation was the absence of RAPL counters, forcing CPU-time proxies for energy and explaining the CodeCarbon divergence; bare-metal `perf stat` or Intel Power Gadget would give direct joules. A secondary point is that some profiler statistics assume `object` columns, so we verified that `category` columns are handled identically (confirmed by the correctness check).

Are the strategies complete? The infographic [7] covers the space well, but our experience suggests adding “**Use efficient data formats**” (between C3 and D6). We *measured* rather than hypothesised this: CSV $\rightarrow$ Parquet cut load time 95% and on-disk size 85% (Section 9), and the storage reduction matters because reports can accumulate faster than runtime energy. This is distinct enough from C3 and D6 to warrant its own entry.

Future work. A documentation pull request upstreaming the lossless dtype/usecols pattern as a best-practice guide; direct RAPL-based energy measurement on bare metal; and extending the study to the Spark backend and to multi-gigabyte inputs where the scaling advantage should widen further.

13 Conclusion

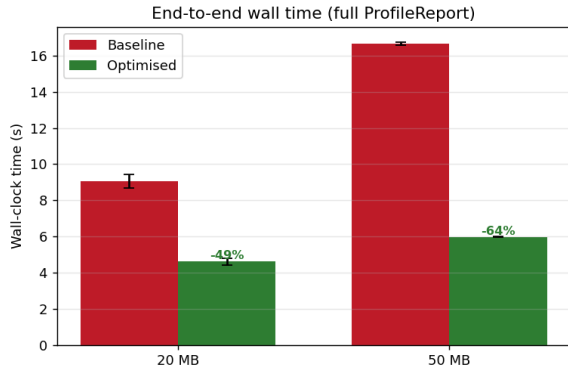
A measured, hypothesis-driven, caller-side intervention reduces fg-data-profiling’s end-to-end time and energy by 49–64% and its load memory by 88%, losslessly and in under ten lines, with the advantage growing on larger inputs. Crucially, full-pipeline measurement reveals that the library’s

correlation and rendering stages cap process memory, bounding what caller-side work can achieve and pointing to configuration tuning, sampling, efficient formats, and caching as complementary levers. The broader lesson generalises to any pandas pipeline: library defaults optimise for safety and generality, not efficiency, and telling the tool the truth about the data upfront is a cheap, transferable green-software win.

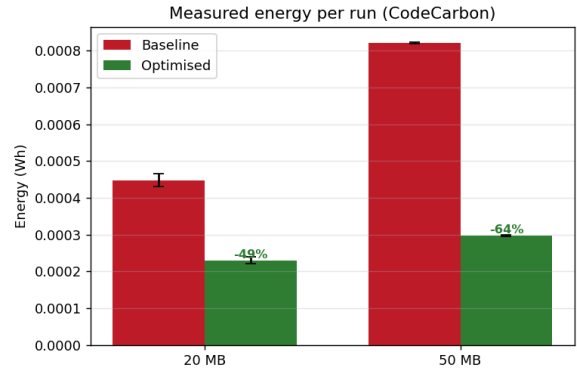
References

- [1] Udit Gupta et al. Chasing carbon: The elusive environmental footprint of computing. In *IEEE HPCA*, 2021. doi: 10.1109/HPCA51647.2021.00010.
- [2] Rui Pereira, Marco Couto, Francisco Ribeiro, Rui Rua, Jácome Cunha, João Paulo Fernandes, and João Saraiva. Energy efficiency across programming languages: How do energy, time, and memory relate? In *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*, pages 256–267, 2017. doi: 10.1145/3136014.3136031.
- [3] Marcus Hähnel, Björn Döbel, Marcus Völz, and Hermann Härtig. Measuring energy consumption for short code paths using RAPL. *ACM SIGMETRICS Performance Evaluation Review*, 40(3):13–17, 2012. doi: 10.1145/2425248.2425252.
- [4] Wes McKinney. Data structures for statistical computing in python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010. doi: 10.25080/Majora-92bf1922-00a.
- [5] Charles R. Harris et al. Array programming with NumPy. *Nature*, 585:357–362, 2020. doi: 10.1038/s41586-020-2649-2.
- [6] Kadan Lottick et al. Energy usage reports: Environmental awareness as part of algorithmic accountability, 2019.
- [7] Bernard van Gastel. Strategies for green software (incl. AI) v2026.02.25. Infographic, Software Energy Lab, Radboud University. <https://sustainablesoftware.info>, 2026.
- [8] Data-Centric AI Community. fg-data-profiling: Automated eda and data profiling for pandas DataFrames. <https://github.com/Data-Centric-AI-Community/fg-data-profiling>, 2024.
- [9] Python Software Foundation. `tracemalloc` — trace memory allocations. <https://docs.python.org/3/library/tracemalloc.html>, 2024.

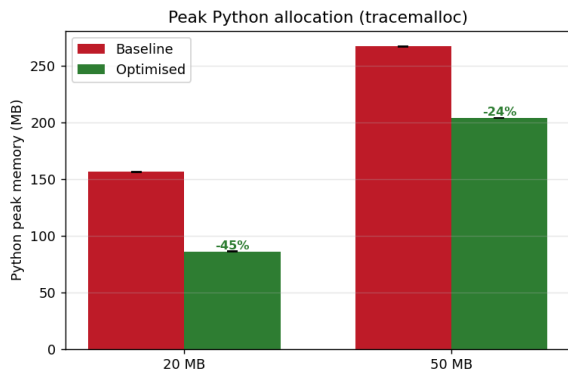
A Additional Plots



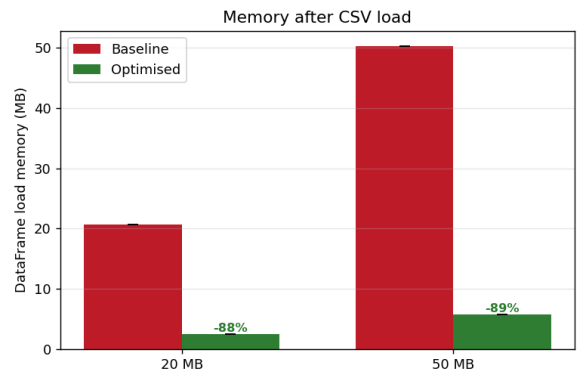
(a) Wall time by size and variant.



(b) Measured energy (CodeCarbon).



(c) Peak Python allocation.



(d) DataFrame load memory.

Figure 6: Supplementary end-to-end measurements across both input sizes.